Numerical Roe Flux Workflow for General Hyperbolic Conservation Laws

This document outlines a numerical procedure to compute the Roe flux for a general 1D hyperbolic conservation law $\mathbf{u}_t + \mathbf{f}(\mathbf{u})_x = 0$, where $\mathbf{u} \in \mathbb{R}^m$ is the conserved variable vector and $\mathbf{f}(\mathbf{u})$ is the flux. The approach is system-agnostic, using numerical approximations to handle arbitrary systems without requiring analytical forms of the equations.

Workflow

1. **Numerical Flux Form**: Compute the Roe flux:

$$\mathbf{F}_{i+1/2} = \frac{1}{2}\left(\mathbf{f}(\mathbf{u}_L) + \mathbf{f}(\mathbf{u}_R)\right) - \frac{1}{2}\sum_{k=1}^{m} |\tilde{\lambda}_k| \tilde{\alpha}_k \tilde{\mathbf{r}}_k$$

   where $\mathbf{u}_L, \mathbf{u}_R \in \mathbb{R}^m$ are left and right states, $\tilde{\lambda}_k$, $\tilde{\alpha}_k$, $\tilde{\mathbf{r}}_k$ are Roe-averaged eigenvalues, wave strengths, and eigenvectors.

2. **Numerical Jacobian**: Approximate the flux Jacobian $\mathbf{A} = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}$ at $\mathbf{u} \approx (\mathbf{u}_L + \mathbf{u}_R)/2$ using finite differences:

$$\mathbf{A}_{ij} \approx \frac{f_i(\mathbf{u} + \epsilon \mathbf{e}_j) - f_i(\mathbf{u} - \epsilon \mathbf{e}_j)}{2\epsilon}$$

   where $\mathbf{e}_j$ is the j-th unit vector, $\epsilon \approx 10^{-6}$.

3. **Eigenstructure**: Compute eigenvalues $\tilde{\lambda}_k$ and eigenvectors $\tilde{\mathbf{r}}_k$ of $\mathbf{A}$ using a numerical eigensolver (e.g., QR algorithm or NumPy).

4. **Wave Strengths**: Solve for $\tilde{\alpha}_k$ in $\Delta\mathbf{u} = \mathbf{u}_R - \mathbf{u}_L = \sum_{k=1}^{m} \tilde{\alpha}_k \tilde{\mathbf{r}}_k$ by projecting:

$$\tilde{\alpha}_k = (\tilde{\mathbf{l}}_k)^T \Delta\mathbf{u}$$

   where $\tilde{\mathbf{l}}_k$ are left eigenvectors.

5. **Assemble Flux**: Compute:

$$\mathbf{F}_{i+1/2} = \frac{1}{2}\left(\mathbf{f}(\mathbf{u}_L) + \mathbf{f}(\mathbf{u}_R)\right) - \frac{1}{2}\sum_{k=1}^{m} |\tilde{\lambda}_k| \tilde{\alpha}_k \tilde{\mathbf{r}}_k$$

6. **Implementation Notes**:
   - Use numerical libraries (e.g., NumPy, SciPy) for matrix operations.
   - Ensure $\mathbf{f}(\mathbf{u})$ is provided as a function.
   - Apply entropy fix: replace $|\tilde{\lambda}_k|$ with $\max(|\tilde{\lambda}_k|, \delta)$, $\delta \approx 10^{-6}$.
   - Test with small ϵ for numerical stability.

Python Implementation

The following Python code implements the numerical Roe flux for a general system, demonstrated with the p-System ($\mathbf{u} = [v, w]^T$, $\mathbf{f} = [w, -\sigma(v)]^T$, $\sigma(v) = v^2$).

```
import numpy as np

def compute_roe_flux(u_L, u_R, flux_func, epsilon=1e-6, delta=1e-6):
    """
    Compute Roe flux for a general 1D hyperbolic conservation law.

    Parameters:
    u_L, u_R : ndarray, left and right state vectors
    flux_func : function, computes flux f(u)
    epsilon : float, perturbation for finite difference Jacobian
    delta : float, entropy fix parameter

    Returns:
    F : ndarray, Roe flux at interface
    """
    m = len(u_L)  # Number of components
    u_avg = 0.5 * (u_L + u_R)  # Simple average for Jacobian
       evaluation

    # Numerical Jacobian
    A = np.zeros((m, m))
    for j in range(m):
        u_plus = u_avg.copy()
        u_minus = u_avg.copy()
        u_plus[j] += epsilon
        u_minus[j] -= epsilon
        A[:, j] = (flux_func(u_plus) - flux_func(u_minus)) / (2 *
           epsilon)

    # Eigenstructure
    eigenvalues, R = np.linalg.eig(A)
    L = np.linalg.inv(R)  # Left eigenvectors

    # Wave strengths
    delta_u = u_R - u_L
    alpha = L @ delta_u

    # Entropy fix
    abs_eigenvalues = np.array([max(abs(lam), delta) for lam in
       eigenvalues])

    # Roe flux
    F_avg = 0.5 * (flux_func(u_L) + flux_func(u_R))
    correction = 0.5 * sum(abs_eigenvalues[k] * alpha[k] * R[:, k]
       for k in range(m))
    F = F_avg - correction
```

```
    return F

# p-System flux function
def p_system_flux(u, sigma_func=lambda v: v**2):
    """
    Flux function for p-System: u = [v, w], f = [w, p(v)], p(v) = -
       sigma(v)
    """
    v, w = u
    return np.array([w, -sigma_func(v)])

# Example usage for p-System
def main():
    # Example states
    u_L = np.array([1.0, 0.5])  # [v_L, w_L]
    u_R = np.array([0.8, 0.3])  # [v_R, w_R]

    # Compute Roe flux
    F = compute_roe_flux(u_L, u_R, p_system_flux)

    print("Left state u_L:", u_L)
    print("Right state u_R:", u_R)
    print("Roe flux F:", F)

if __name__ == "__main__":
    main()
```

This implementation is flexible, relying on numerical approximations and a provided flux function, making it applicable to any hyperbolic conservation law.